

NATIONAL INSTITUTE OF TECHNOLOGY CALICUT

Department of Electronics & Communication Engineering
Microcontroller Lab — EC2092E

DIGITAL IC TESTER Using STM32F401CCU6 Microcontroller

Mini Project Report
Department of Electronics & Communication Engineering

Submitted by

Sooraj K R	B241226EC
Soorya Narayan S	B241229EC
Soupayan Dutta	B241230EC

Academic Year 2025–26

1. Objective

To design and implement a digital IC tester using the STM32F401CCU6 microcontroller capable of verifying the logical functionality of 74HC-series ICs through GPIO-driven truth table testing, with results displayed on an SSD1306 OLED (via I²C), indicated by blue (PASS) and red (FAIL) LEDs, and IC selected via a 4×4 matrix keypad.

2. Introduction

Digital ICs are fundamental building blocks in electronic systems. Faulty or counterfeit ICs often cause hard-to-diagnose failures. An automated IC tester eliminates manual probe-based verification by systematically driving logic inputs and comparing the outputs against a known truth table.

This project utilises the STM32F401CCU6 (BlackPill) microcontroller running at 84 MHz on a 3.3 V supply. The firmware — written in the Arduino framework — manages GPIO driving, I²C OLED communication, keypad scanning, and a debug subsystem entirely through OLED output (no UART dependency). A 40-pin ZIF socket provides tool-free IC insertion and extraction.

3. Components Used

Component	Specification	Qty
Microcontroller	STM32F401CCU6 (BlackPill)	1
OLED Display	SSD1306, 128×64, I ² C (0x3C)	1
Matrix Keypad	4×4, 16-key	1
ZIF Socket	40-pin, DIP	1
LED (PASS)	Blue, 5 mm	1
LED (FAIL)	Red, 5 mm	1
Resistors	330 Ω (IC interface)	12
Resistors	220 Ω (LED current limiting)	2
Decoupling Cap.	100 nF ceramic	1
74HC-series ICs	00, 02, 04, 08, 32, 86, 138, 245	as needed

4. Circuit Description

IC Interface (40-pin ZIF Socket): Pins PA0–PA7 of the STM32 drive the IC input pins via 330 Ω current-limiting resistors. The IC output pins connect to PB0–PB3, configured as GPIO inputs. The VCC pin of the ZIF socket is tied to 3.3 V and GND to ground. A 100 nF decoupling capacitor is placed across VCC and GND for supply noise suppression. The 40-pin ZIF socket accommodates standard DIP packages up to 40 pins, making the tester compatible with a wide range of ICs.

OLED Display (SSD1306) — I²C: The SSD1306 OLED communicates via I²C at address 0x3C. SDA is connected to PB7 and SCL to PB6. The OLED module used includes

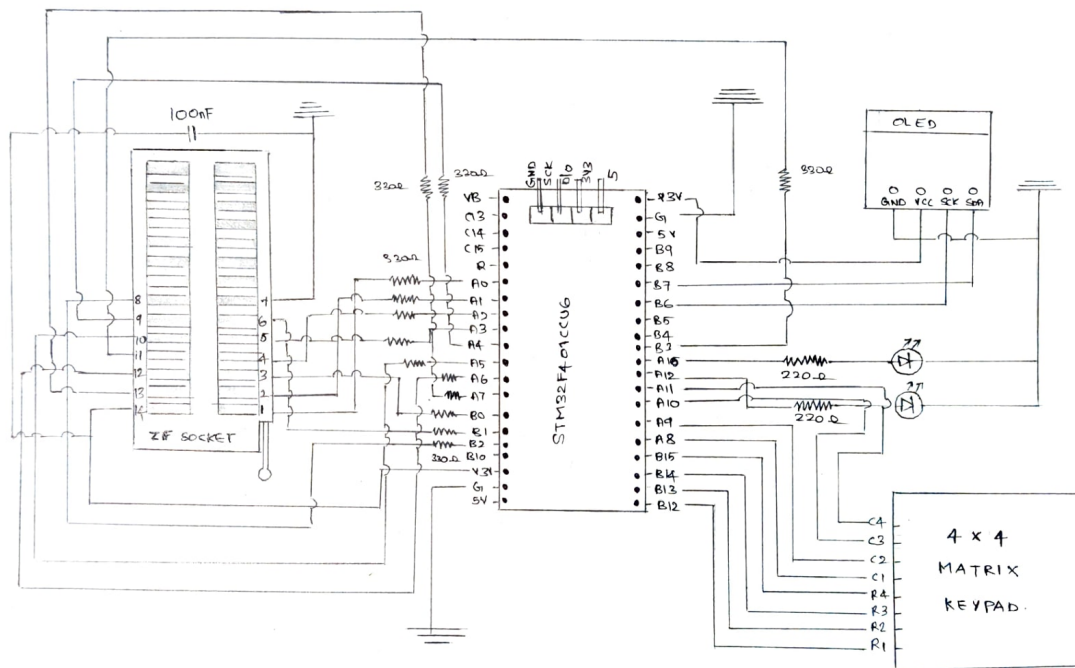


Figure 1: Circuit Diagram — Digital IC Tester using STM32F401CCU6

on-board pull-up resistors on the SDA and SCL lines, so no external pull-ups were required. The `Wire.setSDA(PB7)` and `Wire.setSCL(PB6)` calls in firmware configure the hardware I²C peripheral before initialisation.

4×4 Matrix Keypad: Row lines R1–R4 connect to PB12–PB15 (outputs); column lines C1–C4 connect to PA8–PA11 (inputs with internal pull-ups). Scanning is performed row-by-row using the Keypad library.

Indicator LEDs: The PASS indicator is a blue LED driven from PA12 through a 220 Ω resistor. The FAIL indicator is a red LED driven from PB4 through a 220 Ω resistor (PA15 was avoided due to JTDI conflict).

5. Working Principle

The user places a 74HC-series IC in the ZIF socket, powers the board, and selects the IC type from the OLED menu using the keypad. On pressing #, the firmware:

1. Configures PA0–PA7 as outputs and PB0–PB3 as inputs.
2. Iterates through each truth table row, driving the corresponding input pattern via `driveInputs()`.
3. Reads the IC response via `readOutputs()` after a 500 μs settling delay.
4. Compares the result against the expected value. A mismatch terminates the test immediately.

- Displays PASS or FAIL on the OLED via I²C and lights the corresponding LED (blue = PASS, red = FAIL) for 3 seconds.

6. Algorithm

- Initialise** I²C bus (PB6/PB7), OLED, LEDs, and keypad.
- Blink** both LEDs three times as a hardware self-test.
- Display** the IC selection menu on the OLED.
- Wait** for a keypad press. If key is 1--8, store the selected IC index.
- Show** the selected IC name with options: Test (#), Menu (*), WiringCheck (A).
- On #: configure GPIOs → run `debugFloatCheck()` → enter truth table loop:
 - Drive input pattern via `driveInputs()`.
 - Wait 500 μ s for IC propagation.
 - Read outputs via `readOutputs()`.
 - Call `debugOneRow()` to display the row result on OLED.
 - If mismatch: set FAIL flag, record details, break loop.
- Show result:** illuminate PASS/FAIL LED, display result on OLED for 3s.
- Return** to the main menu. Repeat from Step 3.

7. Source Code

IC Tester Firmware — STM32F401CCU6 (Arduino Framework)

```

1  /*
2   * IC Tester -- STM32F401CCUx (BlackPill)
3   * OLED-DEBUG VERSION -- all debug output goes to OLED, no Serial
4   * dependency
5   */
6  #include <Wire.h>
7  #include <Adafruit_GFX.h>
8  #include <Adafruit_SSD1306.h>
9  #include <Keypad.h>
10
11  // --- OLED
12  -----
13
14  #define OLED_WIDTH  128
15  #define OLED_HEIGHT 64
16  #define OLED_ADDR   0x3C
17
18  Adafruit_SSD1306 display(OLED_WIDTH, OLED_HEIGHT, &Wire, -1);
19  bool oledOK = false;
20
21  // --- PINS

```

```

-----
20 const uint8_t IN_PINS[8] = { PA0, PA1, PA2, PA3, PA4, PA5, PA6,
    PA7 };
21 const uint8_t OUT_PINS[4] = { PB0, PB1, PB2, PB3 };
22
23 #define LED_PASS PA12
24 #define LED_FAIL PB4 // changed from PA15 to avoid JTDI
    conflict
25
26 // --- KEYPAD
    -----
27 const byte ROWS = 4, COLS = 4;
28 char keys[ROWS][COLS] = {
29     {'1','2','3','A'},
30     {'4','5','6','B'},
31     {'7','8','9','C'},
32     {'*','0','#','D'}
33 };
34 byte rowPins[ROWS] = { PB12, PB13, PB14, PB15 };
35 byte colPins[COLS] = { PA8, PA9, PA10, PA11 };
36 Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS,
    COLS);
37
38 // --- IC DATABASE
    -----
39 struct ICTest {
40     const char* name;
41     void (*runTest)();
42 };
43
44 void test_74HC00(); void test_74HC02(); void test_74HC04();
45 void test_74HC08(); void test_74HC32(); void test_74HC86();
46 void test_74HC138(); void test_74HC245();
47
48 const ICTest icList[] = {
49     { "74HC00 NAND", test_74HC00 },
50     { "74HC02 NOR", test_74HC02 },
51     { "74HC04 NOT", test_74HC04 },
52     { "74HC08 AND", test_74HC08 },
53     { "74HC32 OR", test_74HC32 },
54     { "74HC86 XOR", test_74HC86 },
55     { "74HC138 DEC", test_74HC138 },
56     { "74HC245 BUF", test_74HC245 },
57 };
58 const uint8_t IC_COUNT = sizeof(icList) / sizeof(icList[0]);
59 int selectedIC = -1;
60
61 // --- OLED HELPERS
    -----
62 void oledClear() {
63     if (!oledOK) return;
64     display.clearDisplay();
65     display.setCursor(0, 0);
66     display.setTextSize(1);
67     display.setTextColor(SSD1306_WHITE);

```

```

68 }
69
70 void oledPrint(const char* line) {
71     if (!oledOK) return;
72     display.println(line);
73 }
74
75 void oledPrintBig(const char* line) {
76     if (!oledOK) return;
77     display.setTextSize(2);
78     display.println(line);
79     display.setTextSize(1);
80 }
81
82 void oledShow() {
83     if (!oledOK) return;
84     display.display();
85 }
86
87 // Show a full debug screen and wait for keypress to continue
88 void oledDebugScreen(const char* title,
89                     const char* l1 = nullptr,
90                     const char* l2 = nullptr,
91                     const char* l3 = nullptr,
92                     const char* l4 = nullptr,
93                     bool waitKey = true) {
94     oledClear();
95     display.setTextSize(1);
96     display.setTextColor(SSD1306_WHITE);
97     display.setCursor(0, 0);
98     display.println(title);
99     display.drawLine(0, 9, 127, 9, SSD1306_WHITE);
100    display.setCursor(0, 12);
101    if (l1) display.println(l1);
102    if (l2) display.println(l2);
103    if (l3) display.println(l3);
104    if (l4) display.println(l4);
105    if (waitKey) {
106        display.setCursor(0, 56);
107        display.print("* continue...");
108    }
109    display.display();
110    if (waitKey) {
111        while (true) {
112            char k = keypad.getKey();
113            if (k) break;
114        }
115    }
116 }
117
118 // --- RESULT FEEDBACK
119 -----
120 void showResult(bool pass, const char* details = nullptr) {
121     digitalWrite(LED_PASS, pass ? HIGH : LOW);
122     digitalWrite(LED_FAIL, pass ? LOW : HIGH);
123     oledClear();
124     oledPrintBig(pass ? "PASS" : "FAIL");

```

```

124     oledPrint(icList[selectedIC].name);
125     if (details) oledPrint(details);
126     oledShow();
127     delay(3000);
128     digitalWrite(LED_PASS, LOW);
129     digitalWrite(LED_FAIL, LOW);
130 }
131
132 // --- PIN HELPERS
133 -----
134 void setPinModes() {
135     for (auto p : IN_PINS) pinMode(p, OUTPUT);
136     for (auto p : OUT_PINS) pinMode(p, INPUT);
137 }
138
139 void driveInputs(uint8_t vals, uint8_t count = 8) {
140     for (uint8_t i = 0; i < count; i++)
141         digitalWrite(IN_PINS[i], (vals >> i) & 1 ? HIGH : LOW);
142 }
143
144 uint8_t readOutputs(uint8_t count = 4) {
145     uint8_t result = 0;
146     for (uint8_t i = 0; i < count; i++)
147         if (digitalRead(OUT_PINS[i]) == HIGH) result |= (1 << i);
148     return result;
149 }
150
151 // --- OLED DEBUG HELPERS
152 -----
153 // Shows raw OUT_PINS state before any inputs are driven
154 void debugFloatCheck() {
155     uint8_t raw = readOutputs(4);
156     char line1[22], line2[22], line3[22], line4[22];
157     snprintf(line1, sizeof(line1), "PB0=%d PB1=%d",
158             digitalRead(OUT_PINS[0]), digitalRead(OUT_PINS[1]));
159     snprintf(line2, sizeof(line2), "PB2=%d PB3=%d",
160             digitalRead(OUT_PINS[2]), digitalRead(OUT_PINS[3]));
161     snprintf(line3, sizeof(line3), "Raw=0x%X", raw);
162     if (raw == 0x0F)
163         snprintf(line4, sizeof(line4), "WARN:all HIGH!");
164     else if (raw == 0x00)
165         snprintf(line4, sizeof(line4), "OK: all LOW");
166     else
167         snprintf(line4, sizeof(line4), "MIXED state");
168     oledDebugScreen("FloatChk(noIC)", line1, line2, line3, line4,
169                     true);
170 }
171
172 // Shows one truth table row result on OLED
173 void debugOneRow(const char* icName,
174                 uint8_t inputs, uint8_t expected, uint8_t got) {
175     char l1[22], l2[22], l3[22], l4[22];
176     snprintf(l1, sizeof(l1), "IC: %s", icName);
177     snprintf(l2, sizeof(l2), "IN :%d%d%d%d %d%d%d%d",
178             (inputs>>7)&1, (inputs>>6)&1, (inputs>>5)&1, (inputs>>4)
179             &1,

```

```

177         (inputs>>3)&1,(inputs>>2)&1,(inputs>>1)&1,(inputs>>0)
178         &1);
179     snprintf(l3, sizeof(l3), "EXP:%d%d%d%d GOT:%d%d%d%d",
180         (expected>>3)&1,(expected>>2)&1,(expected>>1)&1,(
181         expected>>0)&1,
182         (got>>3)&1,      (got>>2)&1,      (got>>1)&1,      (got>>0)
183         &1);
184     snprintf(l4, sizeof(l4), "%s",
185         (got == expected) ? "ROW: OK" : "ROW: FAIL!");
186     oledDebugScreen("RowDebug", l1, l2, l3, l4, true);
187 }
188
189 // Drives each IN pin HIGH one at a time for wiring verification
190 void debugWiringCheck() {
191     oledDebugScreen("WiringCheck",
192         "Will drive each",
193         "IN pin HIGH one",
194         "by one. Watch",
195         "OUT pins change.", true);
196     for (uint8_t i = 0; i < 8; i++) {
197         for (uint8_t j = 0; j < 8; j++)
198             digitalWrite(IN_PINS[j], (j == i) ? HIGH : LOW);
199         delay(100);
200         uint8_t raw = readOutputs(4);
201         char l1[22], l2[22], l3[22];
202         snprintf(l1, sizeof(l1), "IN_PIN[%d]=PA%d HIGH", i, i);
203         snprintf(l2, sizeof(l2), "OUT raw=0x%X", raw);
204         snprintf(l3, sizeof(l3), "PBO=%d 1=%d 2=%d 3=%d",
205             digitalRead(OUT_PINS[0]), digitalRead(OUT_PINS[1]),
206             digitalRead(OUT_PINS[2]), digitalRead(OUT_PINS[3]));
207         oledDebugScreen("WiringChk", l1, l2, l3, "* = next pin", true
208             );
209     }
210     for (auto p : IN_PINS) digitalWrite(p, LOW);
211 }
212
213 // --- IC TESTS
214 -----
215
216 void test_74HC00() { // NAND: 4 gates, 2 inputs each
217     setPinModes(); debugFloatCheck();
218     bool pass = true; char detail[40];
219     struct { uint8_t inputs; uint8_t expected; } tt[] = {
220         {0b00000000, 0b1111}, {0b01010101, 0b1111},
221         {0b10101010, 0b1111}, {0b11111111, 0b0000},
222     };
223     for (auto& t : tt) {
224         driveInputs(t.inputs); delayMicroseconds(500);
225         uint8_t got = readOutputs(4);
226         debugOneRow("HC00", t.inputs, t.expected, got);
227         if (got != t.expected) {
228             pass = false;
229             snprintf(detail, sizeof(detail), "in=%02X e=%X g=%X",
230                 t.inputs, t.expected, got); break;
231         }
232     }
233     showResult(pass, pass ? nullptr : detail);
234 }

```



```

229 }
230
231 void test_74HC02() { // NOR: 4 gates, 2 inputs each
232     setPinModes(); debugFloatCheck();
233     bool pass = true; char detail[40];
234     struct { uint8_t inputs; uint8_t expected; } tt[] = {
235         {0b00000000, 0b1111}, {0b01010101, 0b0000},
236         {0b10101010, 0b0000}, {0b11111111, 0b0000},
237     };
238     for (auto& t : tt) {
239         driveInputs(t.inputs); delayMicroseconds(500);
240         uint8_t got = readOutputs(4);
241         debugOneRow("HC02", t.inputs, t.expected, got);
242         if (got != t.expected) {
243             pass = false;
244             snprintf(detail, sizeof(detail), "in=%02X e=%X g=%X",
245                 t.inputs, t.expected, got); break;
246         }
247     }
248     showResult(pass, pass ? nullptr : detail);
249 }
250
251 void test_74HC04() { // NOT: 6 inverters, 4 tested
252     setPinModes(); debugFloatCheck();
253     bool pass = true; char detail[40];
254     struct { uint8_t inputs; uint8_t expected; } tt[] = {
255         {0b00000000, 0b1111}, {0b00001111, 0b0000},
256     };
257     for (auto& t : tt) {
258         for (uint8_t i = 0; i < 4; i++)
259             digitalWrite(IN_PINS[i], (t.inputs >> i) & 1 ? HIGH : LOW);
260         delayMicroseconds(500);
261         uint8_t got = readOutputs(4);
262         debugOneRow("HC04", t.inputs, t.expected, got);
263         if (got != t.expected) {
264             pass = false;
265             snprintf(detail, sizeof(detail), "in=%X e=%X g=%X",
266                 t.inputs, t.expected, got); break;
267         }
268     }
269     showResult(pass, pass ? nullptr : detail);
270 }
271
272 void test_74HC08() { // AND: 4 gates, 2 inputs each
273     setPinModes(); debugFloatCheck();
274     bool pass = true; char detail[40];
275     struct { uint8_t inputs; uint8_t expected; } tt[] = {
276         {0b00000000, 0b0000}, {0b01010101, 0b0000},
277         {0b10101010, 0b0000}, {0b11111111, 0b1111},
278     };
279     for (auto& t : tt) {
280         driveInputs(t.inputs); delayMicroseconds(500);
281         uint8_t got = readOutputs(4);
282         debugOneRow("HC08", t.inputs, t.expected, got);
283         if (got != t.expected) {
284             pass = false;
285             snprintf(detail, sizeof(detail), "in=%02X e=%X g=%X",

```

```

286         t.inputs, t.expected, got); break;
287     }
288 }
289 showResult(pass, pass ? nullptr : detail);
290 }
291
292 void test_74HC32() { // OR: 4 gates, 2 inputs each
293     setPinModes(); debugFloatCheck();
294     bool pass = true; char detail[40];
295     struct { uint8_t inputs; uint8_t expected; } tt[] = {
296         {0b00000000, 0b0000}, {0b01010101, 0b1111},
297         {0b10101010, 0b1111}, {0b11111111, 0b1111},
298     };
299     for (auto& t : tt) {
300         driveInputs(t.inputs); delayMicroseconds(500);
301         uint8_t got = readOutputs(4);
302         debugOneRow("HC32", t.inputs, t.expected, got);
303         if (got != t.expected) {
304             pass = false;
305             snprintf(detail, sizeof(detail), "in=%02X e=%X g=%X",
306                     t.inputs, t.expected, got); break;
307         }
308     }
309     showResult(pass, pass ? nullptr : detail);
310 }
311
312 void test_74HC86() { // XOR: 4 gates, 2 inputs each
313     setPinModes(); debugFloatCheck();
314     bool pass = true; char detail[40];
315     struct { uint8_t inputs; uint8_t expected; } tt[] = {
316         {0b00000000, 0b0000}, {0b01010101, 0b1111},
317         {0b10101010, 0b1111}, {0b11111111, 0b0000},
318     };
319     for (auto& t : tt) {
320         driveInputs(t.inputs); delayMicroseconds(500);
321         uint8_t got = readOutputs(4);
322         debugOneRow("HC86", t.inputs, t.expected, got);
323         if (got != t.expected) {
324             pass = false;
325             snprintf(detail, sizeof(detail), "in=%02X e=%X g=%X",
326                     t.inputs, t.expected, got); break;
327         }
328     }
329     showResult(pass, pass ? nullptr : detail);
330 }
331
332 void test_74HC138() { // 3-to-8 decoder, 4 outputs tested
333     setPinModes(); debugFloatCheck();
334     bool pass = true; char detail[40];
335     digitalWrite(IN_PINS[3], HIGH);
336     digitalWrite(IN_PINS[4], LOW);
337     digitalWrite(IN_PINS[5], LOW);
338     delayMicroseconds(500);
339     uint8_t expected[4] = { 0b1110, 0b1101, 0b1011, 0b0111 };
340     for (uint8_t addr = 0; addr < 4; addr++) {
341         digitalWrite(IN_PINS[0], (addr >> 0) & 1);
342         digitalWrite(IN_PINS[1], (addr >> 1) & 1);

```

```

343     digitalWrite(IN_PINS[2], (addr >> 2) & 1);
344     delayMicroseconds(500);
345     uint8_t got = readOutputs(4);
346     debugOneRow("HC138", addr, expected[addr], got);
347     if (got != expected[addr]) {
348         pass = false;
349         snprintf(detail, sizeof(detail), "addr=%d e=%X g=%X",
350                 addr, expected[addr], got); break;
351     }
352 }
353 showResult(pass, pass ? nullptr : detail);
354 }
355
356 void test_74HC245() { // Octal bus transceiver, 4 lines tested
357     setPinModes(); debugFloatCheck();
358     bool pass = true; char detail[40];
359     digitalWrite(IN_PINS[0], LOW); // DIR = A->B
360     digitalWrite(IN_PINS[1], LOW); // OE = enabled
361     delayMicroseconds(500);
362     uint8_t testVals[] = { 0b0000, 0b1010, 0b0101, 0b1111 };
363     for (uint8_t v : testVals) {
364         for (uint8_t i = 0; i < 4; i++)
365             digitalWrite(IN_PINS[i + 2], (v >> i) & 1 ? HIGH : LOW);
366         delayMicroseconds(500);
367         uint8_t got = readOutputs(4);
368         debugOneRow("HC245", v, v, got);
369         if (got != v) {
370             pass = false;
371             snprintf(detail, sizeof(detail), "in=%X e=%X g=%X", v, v,
372                     got); break;
373         }
374     }
375     showResult(pass, pass ? nullptr : detail);
376 }
377 // --- MENU
378
378 void showMenu() {
379     if (!oledOK) return;
380     display.clearDisplay();
381     display.setTextSize(1);
382     display.setTextColor(SSD1306_WHITE);
383     display.setCursor(0, 0);
384     display.println("=== IC Tester ===");
385     for (uint8_t i = 0; i < IC_COUNT; i++) {
386         char line[22];
387         snprintf(line, sizeof(line), "%d:%s", i + 1, icList[i].name);
388         display.println(line);
389     }
390     display.println("#=Run *=Back");
391     display.display();
392 }
393
394 void showSelected() {
395     if (!oledOK) return;
396     display.clearDisplay();

```

```
397     display.setTextSize(1);
398     display.setTextColor(SSD1306_WHITE);
399     display.setCursor(0, 0);
400     display.println("Selected:");
401     display.println(icList[selectedIC].name);
402     display.println("");
403     display.println("# = Test");
404     display.println("* = Menu");
405     display.println("A = WiringCheck");
406     display.display();
407 }
408
409 // --- SETUP
410
411 void setup() {
412     Wire.setSDA(PB7);
413     Wire.setSCL(PB6);
414     Wire.begin();
415     Wire.setClock(100000);
416
417     pinMode(LED_PASS, OUTPUT);
418     pinMode(LED_FAIL, OUTPUT);
419     digitalWrite(LED_PASS, LOW);
420     digitalWrite(LED_FAIL, LOW);
421
422     if (display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
423         oledOK = true;
424         display.clearDisplay();
425         display.setTextSize(1);
426         display.setTextColor(SSD1306_WHITE);
427         display.setCursor(0, 0);
428         display.println("IC Tester");
429         display.println("DEBUG MODE");
430         display.println("OLED OK");
431         display.display();
432         delay(1000);
433     }
434
435     for (int i = 0; i < 3; i++) {
436         digitalWrite(LED_PASS, HIGH); delay(150);
437         digitalWrite(LED_PASS, LOW);
438         digitalWrite(LED_FAIL, HIGH); delay(150);
439         digitalWrite(LED_FAIL, LOW);
440     }
441
442     showMenu();
443 }
444
445 // --- LOOP
446
447 void loop() {
448     char key = keypad.getKey();
449     if (!key) return;
450
451     if (key >= '1' && key <= ('0' + IC_COUNT)) {
```

```

450     selectedIC = key - '1';
451     showSelected();
452
453 } else if (key == '#') {
454     if (selectedIC < 0) {
455         oledDebugScreen("Error",
456                         "No IC selected!",
457                         "Press 1-8 first",
458                         nullptr, nullptr, false);
459         delay(1500);
460         showMenu();
461     } else {
462         oledClear();
463         oledPrint("Testing...");
464         oledPrint(icList[selectedIC].name);
465         oledShow();
466         delay(500);
467         icList[selectedIC].runTest();
468         delay(300);
469         showMenu();
470     }
471
472 } else if (key == 'A') {
473     if (selectedIC >= 0) {
474         setPinModes();
475         debugWiringCheck();
476         showSelected();
477     }
478
479 } else if (key == '*') {
480     selectedIC = -1;
481     showMenu();
482 }
483 }

```

8. Code Explanation

GPIO Architecture. PA0–PA7 are declared as OUTPUT pins to drive IC inputs via 330 Ω resistors. PB0–PB3 are declared as floating INPUT pins, relying on the IC's own output drive strength. Pin modes are freshly applied before every test via `setPinModes()`, ensuring a clean state regardless of any prior test.

IC Database and Function Pointers. An array of `ICTest` structs pairs each IC's name string with a dedicated test function pointer. All eight ICs — 74HC00, 02, 04, 08, 32, 86, 138, and 245 — are forward-declared and registered in `icList[]`. The menu, dispatcher, and result display all index this single array, making it trivial to add further ICs.

Truth Table Testing — `driveInputs()` / `readOutputs()`. `driveInputs(vals)` unpacks the 8-bit `vals` argument bit-by-bit onto PA0–PA7. `readOutputs()` assembles PB0–PB3 into a 4-bit result. A 500 μ s delay between driving and reading allows IC propagation settling. Each IC test iterates through its truth table, calls `debugOneRow()` for every row, and breaks immediately on the first mismatch.

OLED Interface (I²C). The SSD1306 communicates over I²C using the Adafruit library.

`Wire.setSDA(PB7)` and `Wire.setSCL(PB6)` configure the hardware I²C peripheral before initialisation. `oledDebugScreen()` renders a titled, underlined screen with up to four text lines and optionally blocks until a keypad key is pressed, enabling step-by-step inspection.

Menu System and `showSelected()`. Two distinct display routines are used: `showMenu()` lists all eight ICs with their index keys, while `showSelected()` shows the chosen IC name along with the available actions (Test, Menu, WiringCheck). The main loop dispatches on the pressed key — digits 1–8 select an IC, # runs the test, * returns to the menu, and A triggers the wiring diagnostic, after which `showSelected()` is called again.

Keypad Scanning. The Keypad library performs row-by-column scanning on PB12–PB15 (rows) and PA8–PA11 (columns). `keypad.getKey()` is non-blocking; the main loop polls it continuously and acts only when a valid key is returned.

8.1. Debug Features

`debugFloatCheck()` reads all four output pins before any inputs are driven. An all-HIGH reading warns of floating or pull-up issues; all-LOW confirms the correct idle state. Result is displayed on the OLED and pauses for a keypress.

`debugOneRow()` displays the IC name, the full 8-bit input pattern, the expected 4-bit output, the actual 4-bit output, and a ROW OK / ROW FAIL verdict on the OLED after each truth table row. Execution pauses for a keypress, enabling step-by-step verification.

`debugWiringCheck()` drives each of the eight input pins HIGH in turn while keeping the rest LOW, then displays the raw output register on the OLED. This allows physical continuity verification without any external equipment. After all eight pins are cycled, all inputs are restored LOW.

9. Applications

- **PCB repair and QA** — rapid go/no-go verification of 74HC ICs before board assembly.
- **Electronics labs** — teaching truth tables and logic gate behaviour interactively.
- **Procurement inspection** — detection of counterfeit or out-of-spec ICs.
- **Field servicing** — portable tester powered via USB without laptop or UART dependency.
- **Expandability** — the 40-pin ZIF socket and modular IC database can accommodate a wide range of DIP packages with minor firmware additions.

10. Conclusion

A compact and reliable Digital IC Tester was successfully designed and implemented on the STM32F401CCU6 platform. The system correctly identifies PASS/FAIL conditions for eight 74HC-series ICs by driving all relevant input combinations and comparing outputs against hard-coded truth tables. A 40-pin ZIF socket enables tool-free IC insertion. The OLED, driven over I²C, provides a fully standalone debug interface with float checking, row-by-row inspection, and wiring verification — eliminating the need for serial monitors during development and field use. Blue and red LEDs provide immediate visual PASS/-FAIL indication. The modular firmware architecture using function pointers facilitates

straightforward extension to additional IC types, demonstrating effective integration of GPIO control, I²C communication, matrix keypad interfacing, and embedded C++ on an ARM Cortex-M4 microcontroller.

11. References

1. STMicroelectronics, *STM32F401xC/xE Reference Manual* (RM0368), Rev. 5, 2020.
2. STMicroelectronics, *STM32F401CC Datasheet*, Rev. 8, 2019.
3. Texas Instruments, *74HC/HCT Logic Family Datasheet Collection*, 2022.
4. Solomon Systech, *SSD1306 OLED Driver Datasheet*, Rev. 1.1, 2008.
5. Adafruit Industries, *Adafruit SSD1306 Library*, https://github.com/adafruit/Adafruit_SSD1306, 2024.